

SOFTWARE DESIGN DOCUMENT (SDD)

TerraVista Plant Disease Detection System using YOLOv8, Gemini LLM, and Flask

1. Introduction

1.1 Purpose of the Document

This Software Design Document provides a detailed description of the architecture, components, data flow, interfaces, algorithms, and overall design of the *Plant Disease Detection System*. This system leverages **deep learning (YOLOv8)** and **LLM-based advisory generation (Google Gemini)** to analyze plant leaf images and return disease predictions with treatment recommendations.

The document aims to serve as a reference for:

- Developers integrating or extending the system
 - Professors or evaluators assessing technical correctness
 - Future maintainers who need insight into architecture
-

1.2 Scope of the Project

The system performs:

1. **Image Acquisition** (Frontend uploads image)
2. **Leaf Disease Classification** using YOLOv8 classifier
3. **Confidence extraction** from neural network outputs
4. **Treatment generation** using Gemini LLM
5. **Standardized JSON response**
6. **UI display** of disease, confidence, and treatment

The system focuses on **Apple leaf diseases** but is expandable to any crop dataset.

2. System Overview

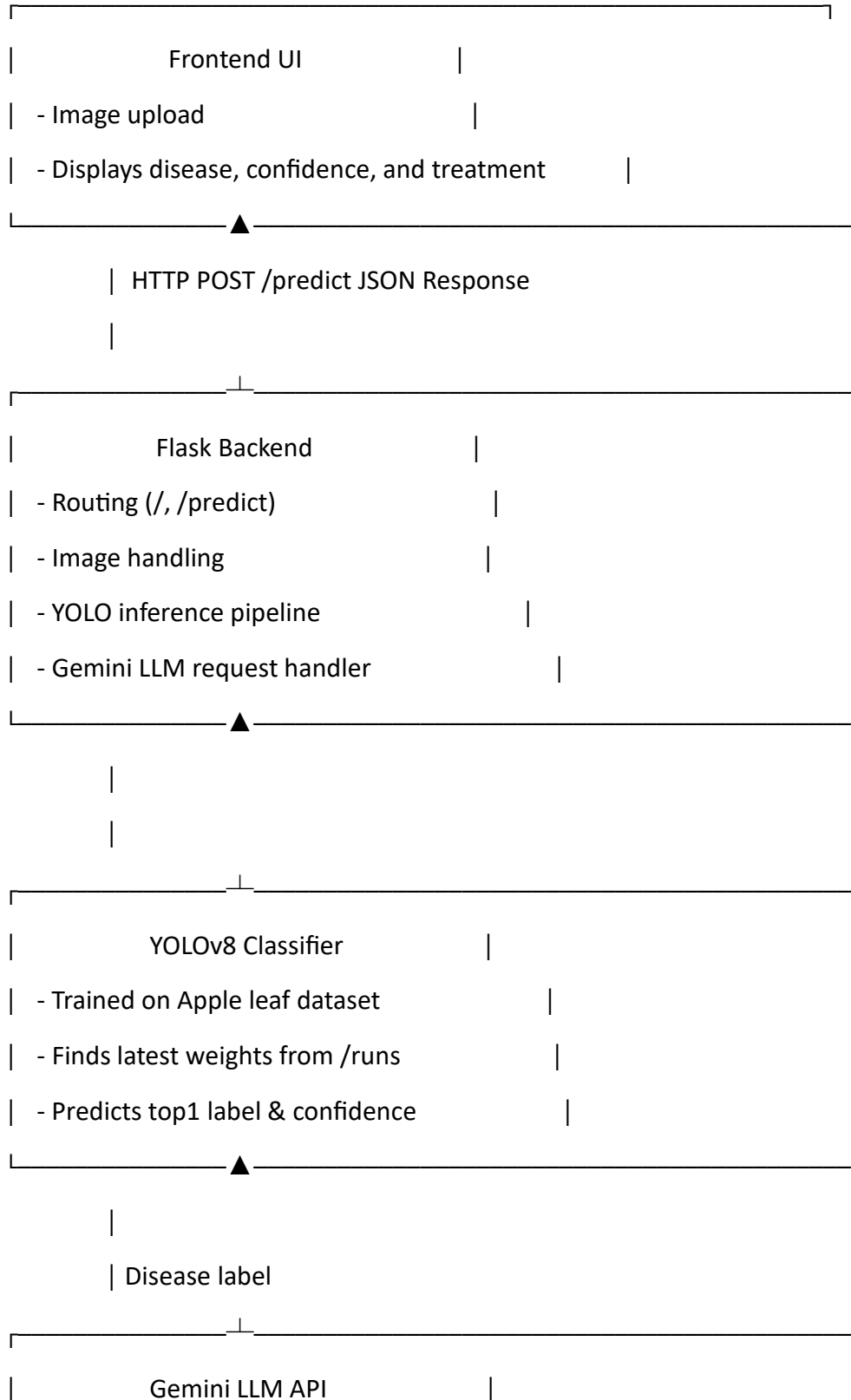
The application uses a **client-server model**:

- **Frontend (HTML/JS)** → Uploads image to API
- **Backend (Flask)** → Runs ML inference
- **YOLOv8 classification model** → Identifies disease

- **Gemini LLM** → Provides treatment steps
 - **REST API** → Bridges frontend and backend
-

3. Overall System Architecture

3.1 Architecture Diagram



	- Prompt structured with detected disease	
	- Generates: Causes, Organic & Chemical solutions,	
	Prevention	

4. Detailed Component Design

4.1 Flask Backend Design

Responsibilities

- Serve the HTML file
- Store and preprocess incoming images
- Perform inference using YOLO
- Communicate with Gemini API
- Return structured JSON response

Routes

Route	Method	Description
/	GET	Returns frontend HTML page
/predict	POST	Returns disease label, confidence, and advice

Key Modules

a. Image Handler

- Accepts file from request
- Converts bytes → PIL.Image → NumPy array
- Ensures compatibility with YOLO input

b. YOLO Inference Module

- Loaded once during app start → avoids reloading every request
- Uses:
 - `model.predict(img, imgsz=224)`
- Extracts:
 - `label = r.names[r.probs.top1]`
 - `confidence = r.probs.top1conf`

c. Gemini Integration Module

- Prompt engineered for:

- Causes
- Organic solutions
- Chemical solutions
- Prevention
- Handles failures gracefully

d. Error Handling

Handled for:

- Missing files
- Wrong image format
- YOLO errors
- Gemini unreachable

4.2 YOLOv8 Model Design

Model Architecture (YOLOv8n-cls)

Backbone: CSPDarknet-n

Neck: SPPF

Head: Classification (fully connected)

Training Pipeline

1. Dataset directory structure:

Apple/

├─ train/

├─ val/

└─ test/

2. Hyperparameters:

- **Epochs:** 30
- **Batch Size:** 16
- **Image Size:** 224
- **Optimizer:** AdamW

3. Checkpoints stored at:

runs/apple_train/weights/best.pt

Model Selection Mechanism

- `find_latest_weights()` → selects latest run
- Tries `best.pt` → otherwise `last.pt`

This allows:

- Multiple experiments
 - Automatic loading
-

4.3 Gemini LLM Module Design

Model Used

- `gemini-2.5-flash`

Why Gemini?

- Fast response
- Good at structured advice
- Low cost compared to other LLMs

Prompt Template

Apple leaf disease: {label}.

Give brief treatment in bullets: causes, organic/chemical solutions, prevention.

Be concise.

Output Structure Example

- 🌿 Cause
 - 🧴 Organic solutions
 - 🧪 Chemical solutions
 - 🛡️ Prevention
-

4.4 Frontend Design

Components

- File input
- Image preview
- Loader animation
- Disease and confidence display
- Advice box

User Flow

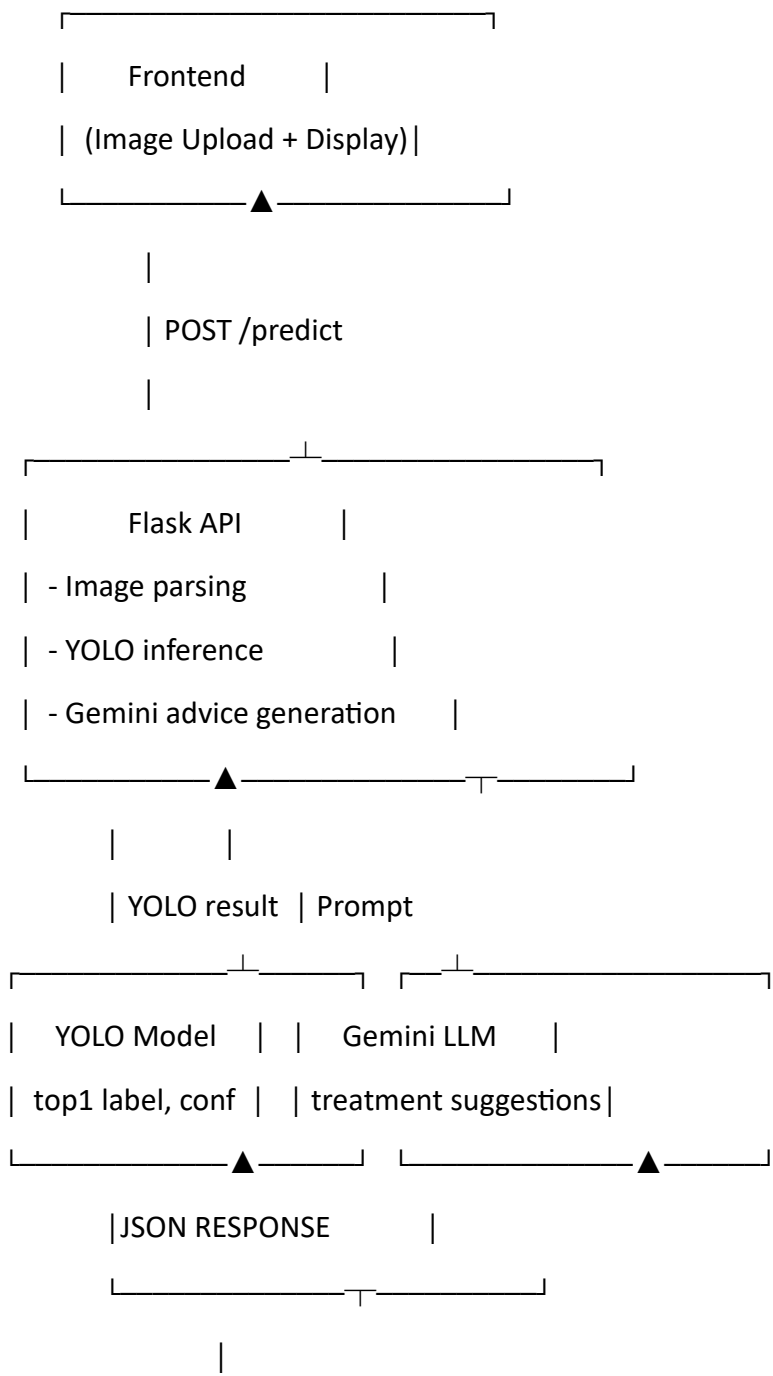
1. User uploads image
2. JavaScript sends FormData to backend
3. Receives JSON
4. UI updates

5. Data Flow Design

Level-0 DFD

User → Upload Image → Flask API → YOLO → Gemini → JSON Output → User

Level-1 DFD



6. Algorithm Design

6.1 YOLO Classification Algorithm

Steps:

- 1. Read image
 - 2. Convert to array
 - 3. Resize to 224×224
 - 4. Normalize pixels
 - 5. Forward pass through CNN
 - 6. Softmax output → class probabilities
 - 7. Return index of highest probability
-

6.2 LLM Treatment Generation Algorithm

- 1. Insert YOLO disease label into prompt template
 - 2. Call generate_content()
 - 3. Parse response text
 - 4. Remove unsafe/invalid outputs if needed
 - 5. Return advice
-

7. Error Handling Design

Backend Errors

Error	Handling Strategy
Invalid image	Return "Invalid file"
YOLO model load failure	Fail fast
Gemini network error	Return fallback notification
Empty result	"No disease detected"

Frontend Errors

- Show red error banner
 - Disable buttons during processing
-

8. Security Design

- Gemini API stored in variable (should be environment variable in production)
 - CORS enabled for frontend
 - Max image file size should be enforced
 - Sanitized file access
-

9. Deployment Design

Development Environment

- python app.py
- Auto opens browser
- Debug mode ON

Production Environment

- Use **Gunicorn + Nginx**
 - Debug mode OFF
 - API key stored in environment variables
 - Rate limiting if exposed publicly
-

10. Hardware/Software Requirements

Hardware

- RAM: 8GB minimum (training needs 12GB+)
- GPU optional for inference

Software

- Python 3.10+
 - YOLOv8 (ultralytics)
 - Flask
 - Google Generative AI SDK
-

11. Future Enhancements

1. Add **segmentation models** to highlight infected region
2. Add **multi-crop support**
3. Drone / Real-time inference

4. Add a **RAG system** for knowledge base
 5. Add disease severity scoring
 6. Mobile app packaging (Flutter)
-

12. Conclusion

This system demonstrates efficient integration of:

- Deep learning classification
- LLM-based agro-advisory
- Web-based interaction
- Automated model selection

The architecture is modular, scalable, and easily maintainable, supporting future expansions into a complete agricultural assistant.